

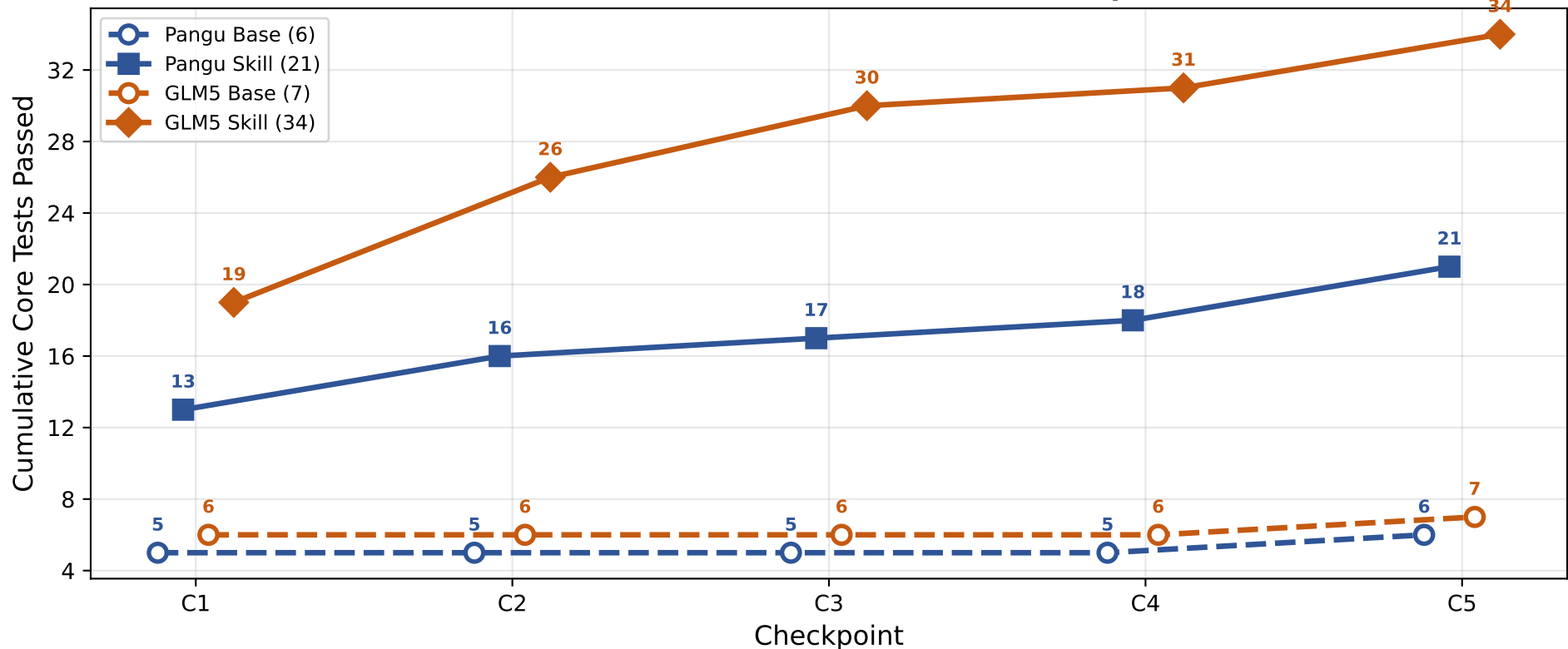
Case Study: pwd_manager

How Code Architecture Drives Benchmark Performance Under Iterative Spec Refinement

Problem: pwd_manager is an interactive CLI password vault with encryption, search, categories, clipboard, password generation, and auto-lock — built across 5 checkpoints.

Checkpoint	Pangu Base	Pangu Skill	GLM5 Base	GLM5 Skill
C1	5/23	13/23	6/23	19/23
C2	0/9	3/9	0/9	7/9
C3	0/6	1/6	0/6	4/6
C4	0/3	1/3	0/3	1/3
C5	1/6	3/6	1/6	3/6
Cumulative	6	21 (+15)	7	34 (+27)

Cumulative Core Tests Passed Across Checkpoints



Checkpoint 1: Code Architecture Comparison

Baseline uses flat functions; Skill runs produce clean OOP with proper encapsulation

← FLAT / MIXED → OOP / ENCAPSULATED →

Pangu Base — 428 lines, 5/23 core

Architecture: FLAT FUNCTIONS

```
class VaultError(Exception)    # error only
class AuthError(VaultError)    # error only

def generate_salt()            # loose function
def derive_key(password, salt) # loose function
def load_vault(key)            # loose function
def setup_vault()              # loose function
def main_menu(key, vault)      # raw dict passed around
```

Pangu Skill — 680 lines, 13/23 core

Architecture: OOP WITH MANAGER PATTERN

```
class Secret:                  # data model

class Vault:                   # collection + serialization

class VaultManager:           # encapsulates ALL operations
    - derive_key(), setup_vault(), unlock()
    - save_vault(), load_vault(), add_secret()

def run_setup() -> VaultManager # clear flow
def run_unlock(manager)         # separated concerns
```

VAULT_DIR = Path('/workspace/.vault') # HARDCODED

GLM5 Base — 652 lines, 6/23 core

Architecture: MIXED (data classes + loose functions)

```
class VaultConfig:    # data only
class Secret:         # data only
class Vault:          # data + some methods

def get_vault_paths() # tries home then cwd
def run_setup(vault)  # vault passed as arg
def run_unlock(vault) # no encapsulation
def add_secret_menu() # inline prompts
```

VAULT_DIR = Path.home() / '.vault' # hardcoded path

VAULT_DIR = env.get('PWD_MANAGER_VAULT_DIR') # CONFIGURABLE

GLM5 Skill — 551 lines, 19/23 core

Architecture: CLEAN OOP, SINGLE RESPONSIBILITY

```
class VaultConfig:    # config management
class VaultStore:     # encryption + persistence
class PasswordManager: # ALL interaction logic
    - setup(), unlock(), main_menu()
    - add_secret(), search(), view_secret()
    - hide_input(), get_input() # consistent I/O

def main():           # just creates PasswordManager
```

self.vault_dir = Path.home()/.vault' # in class init

How Architecture Decisions Map to Test Outcomes

Specific code patterns determine which tests pass or fail

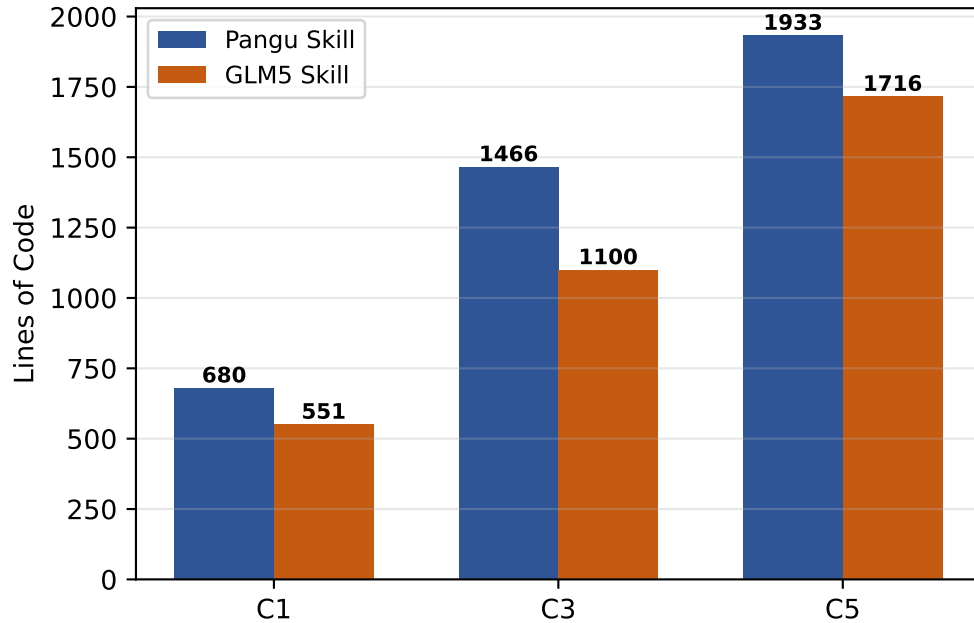
Architecture Pattern	Test Case	Results	Code Difference
Configurable vault directory (env var vs hardcoded path)	test_first_run_creates_default_vault_directory	PB PS GB GS	GS : os.environ.get('PWD_MANAGER_VAULT_DIR') Base: VAULT_DIR = Path('/workspace/.vault')
VaultManager class tracks state (setup → config → vault lifecycle)	test_setup_creates_config_and_unlocks	PB PS GB GS	GS : VaultManager.setup_vault() creates config+vault atomically Base: separate functions, state passed via raw dicts
Separated setup / unlock flows (distinct code paths)	test_missing_config_blocks_unlock	PB PS GB GS	GS : run_setup() vs run_unlock() as separate functions Base: single flow with if/else branching
Salt displayed during setup (confirmation message)	test_setup_confirmation_mentions_salt_value	PB PS GB GS	GS : print(f'Config saved. Salt: {salt}')
Empty state handling (('Empty!' message when no items))	test_all_with_no_items_prints_empty_marker	PB PS GB GS	GS : if not secrets: print('Empty!')
Encapsulated input methods (consistent prompt formatting)	test_item_count_in_menu_prompt	PB PS GB GS	GS : PasswordManager.get_input() with formatted prompt Base: raw input() calls with inconsistent formatting

Pattern: Skill runs consistently use OOP encapsulation → tests that check state management, prompt formatting, and flow separation pass. Base runs use flat functions → these tests fail.

Code Evolution: How C1 Architecture Enables C3-C5 Extensions

OOP design in C1 allows new features as modular additions, not rewrites

Code Growth Across Checkpoints



New Classes Added per Checkpoint

Pangu Skill Evolution:

C1: Secret, Vault, VaultManager (core OOP)
C3: + Category class
+ copy_to_clipboard(), reveal_password()
+ generate_password(), search completion
+ category_management() menu
C5: + check_auto_lock(), export/import
+ parse_args() for CLI flags
+ erase_vault() command

GLM5 Skill Evolution:

C1: VaultConfig, VaultStore, PasswordManager
C5: + ClipboardManager (clipboard ops)
+ PasswordGenerator (password creation)
+ TabCompleter (search autocomplete)
+ InactivityTimer (auto-lock)
Each new feature = new class

Why OOP Architecture Scales Better

Base (Flat) → Hard to Extend

C1: 20+ loose functions, raw dicts
C2: Add more functions → name collisions
C3: Need categories → rewrite data model
C4: Need export → touch every function
C5: Need auto-lock → global state mess

Each new feature ENTANGLES with existing code
→ Regression risk increases per checkpoint

Skill (OOP) → Modular Extension

C1: VaultManager owns all vault ops
C2: Add methods to VaultManager
C3: Add Category class (isolated module)
C4: Add export_vault() method
C5: Add InactivityTimer class (new module)

Each new feature is an ISOLATED addition
→ Existing tests continue to pass

↑ This is why Skill runs maintain performance across checkpoints ↑

Pangu Base
C1→C5: 6 cumulative core
(stuck at 0 after C1)

GLM5 Skill
C1→C5: 34 cumulative core
(steady gains each ckpt)

Key Findings

pwd_manager demonstrates how architectural quality compounds across iterative spec changes

1. Skill runs produce better initial architecture

Both Pangu and GLM5 skill runs use OOP (classes, encapsulation) while baseline runs use flat functions. The skill prompt's 3-phase structure (Audit → Safety Check → Apply) encourages the model to think about code structure before writing.

2. OOP architecture in C1 enables modular extension in C2-C5

GLM5 Skill adds ClipboardManager, PasswordGenerator, TabCompleter, InactivityTimer as separate classes in later checkpoints. Each is a clean module that doesn't touch existing code. Baseline's flat functions require invasive changes for new features.

3. Path dependence: flat C1 → cascading failures

Pangu Base scores 5/23 in C1, then 0 in C2-C4. The flat architecture from C1 can't accommodate new features without breaking existing functionality. This is the core insight of SlopCodeBench: early decisions compound across checkpoints.

4. GLM5 benefits more from skill (+27) than Pangu (+15)

GLM5 Skill produces the cleanest architecture (551 lines, 3 focused classes vs Pangu Skill's 680 lines, 7 classes). More focused classes → better separation of concerns → more tests pass. GLM5's stronger OOP instinct amplifies the skill benefit.

5. Code volume ≠ code quality

GLM5 Skill (551 lines) outperforms GLM5 Base (652 lines) and Pangu Skill (680 lines). The most compact implementation passes the most tests — because every line serves a purpose and the architecture makes the spec's requirements naturally expressible.
